Introduction

Cool. Hello everyone, and welcome to CME 295-- Transformers and Large Language Models. So my name is Afshine. And I will be teaching this class with Shervine, who's in the back. And before I start, I'm just going to introduce ourselves. So we're twin brothers, and we actually had a similar background. So we both went to a school in France called Centrale Paris, and then we each went our way. So on my end, I went to MIT, and then Shervine went to Stanford to do the ICME Master's program. And after that, I guess our industry background is very similar as well. So I first went to Uber, and then Shervine came to Uber as well, and then Shervine left to Google, and I went to Google. And then very recently, I joined Netflix, and Shervine joined Netflix as well. And we've been working on Large Language Models. So yeah, I guess we have technical backgrounds, and mostly oriented towards LLMs.

OK, so why are we doing this class? So since 2020, Shervine and I have been specializing in NLP. And we've been giving this class in the format of a workshop that was done on a yearly basis. So in 2021, 2022, 2023, 2024, ChatGPT came in 2022, and suddenly there was a lot of interest for LLMs. And so it's actually last spring that we started to offer this class as a Stanford course that is now called CME 295, and this is the second instance.

Cool. So what can you expect from this class? So first of all, LLMs are basically everywhere now. And I guess our goal here is twofold. So the first one is to learn about the underlying mechanism that makes all this work. And we're going to see the transformer, which is the foundational architecture that makes all this work. And then the second thing is to know how these LLMs are trained and where they are applied.
So in case you're still wondering if this class is good for you, I would say that this class is great for people who just in general have an interest in this field, either because you wanted to make it your career goal, if you want to be a research scientist or an ML scientist, or if you want to develop a personal project that relies on LLMs to some extent, to just knowing the caveats, I guess what works, what doesn't, or just say if you're in a separate field and you just want to know how this whole AI, GenAI, LLMs thing works and how you can apply it to your domain. So now in terms of prerequisites, I would say that at a very minimum you should have some foundations in ML, like basically know how a model is trained, what a neural network

is, and also some basics in linear algebra, so basically how matrices are multiplied, for instance. But even if you have a developing, I guess, competency in this field, I guess it's fine, we're still be here to help you out. But I guess this is like the ideal set of prerequisites.

Class Logistics

Cool. So still on the logistics, so this class will be held every Friday from 3:30 to 5:20, and it will be held here. So this class is two units. And you have the choice to either take it as a letter or a credit/non-credit. So as you could tell from the setup, we're basically recording this class. And if you cannot for some reason attend this time, this slot, we'll make sure with Shervine to make the recordings available, either tonight, like every Friday night, or on Saturday.

So in terms of the grades, so what we're doing for this quarter is to have two exams. So one is the midterm, which will be happening during our fifth instance, which is October 24. And then the second exam will be the final exam, which will be held in the week of December 8. So date is still TBD. So we'll let you know.

Cool. So every time we have a lecture, we'll be posting the slides and the recordings on the website. And in case you're interested, we also have the syllabus in there, so you can know a little bit what are the topics that we'll be talking about. And the class textbook is this Super Study Guide-- Transformer LLMs. So we have a copy here in case you want to take a look. So yeah, I guess a lot of the concepts that we have in this class will actually be in the book. So I guess it's a helpful way to follow this as well. And also, we did some very short, condensed version of this whole class that we called the VIP cheat sheet. So this one is available on GitHub in case you're interested. And yeah, we also translated it into a number of languages now. By the way, if your language is not there, let us know, and happy to work on that as well together. OK, cool.

I think it's the last things on the logistics part. So in terms of announcements, we'll be posting things on Canvas. In case you have any questions, you can of course reach out to us. But there is also a tab on Canvas that's called Ed. I'm sure you're familiar. So you just click on that, just post your question, and then Shervine and I will

be responding. And I guess to reach out to us, you have this mailing list. Or we're just two, so just ding us.

Cool. So on the logistics, do we have any questions so far? And one thing I forgot to mention is that, given that we're recording this class, I guess if you're asking a question, it may not be super clear for the viewer what your question was. So I'm going to make an effort to just repeat your question. It will sound weird, but I'll try to not forget, but yeah. So any questions so far on the logistics? Yeah.

So the question is whether there are coding parts in the exams. So the answer is no. So the exams will purely focus on concepts that we see in class. And actually, it's not meant to trap you. So I guess if you follow the class, if you see the slides and the concepts that we see, should be fine. Yeah.

Question is, if you're waitlisted, what do you do? I think, so by experience, a lot of people will finalize their schedule. Some people will drop, some won't. In case you're still waitlisted, come talk to us. But I'm pretty confident it's going to be OK, because I think the waitlist right now is six. So, I think it should be fine. Cool. Yeah.
They will be on the website, and we'll make sure to also post a link on Canvas. Yeah. So the question was, where are the slides. And they're on the website. Cool, yeah.

So question is on the weighting of the exams. So there is no homework. So 50% is midterm, 50% is final. And no grades-- I mean, no weights are from that. And particular, I mean if this slot is conflicting with something, just keep in mind that we are recording this. So it's fine if you cannot attend this session.

Oh, is the question that the final is about just the second half of the class? We have not written the exam yet, but I think this is something we are thinking of. So the final is probably going to be about the second half of the topics. Cool. OK, long story short, 50% midterm, 50% final exam. And it's a fun class. Cool. So with that, I'm going to just slowly start the class.

NLP Overview

So another thing that I want to mention was every time we're talking about something, you will see that at the bottom of the slide there will be a source, and it's mostly for-- so first, to credits, whatever, we're quoting, but also for you to dig into

those materials a bit more in case you're interested. Because, of course, we have only two hours per week, and we only have nine or 10 weeks, so there's nowhere near enough time for us to cover everything. And the second disclaimer is you will see that the field is full of abbreviations. So I myself was completely scared of them when I started. But hopefully by the end of the class, you will have a mental mapping of what these abbreviations mean, respect to what they correspond to. So yes, so if you have a mental mapping towards the end of the class, then we know e did a good job.

So with that, let's start. And I guess we will start at a very high level, because I will just assume that I guess we're starting from scratch. And we're going to talk about NLP in general. So NLP is going to be our first abbreviation. So NLP stands for Natural Language Processing. And it is a field that is around manipulating text, just computing things with text. And at a very high level, can basically classify NLP tasks into three buckets. So the first bucket is what we call classification. So we have an input text as an input. And then what we want is to predict something. So one example is you have a movie review and you want to predict whether the sentiment is positive, negative, or neutral. So that's one example. You can also have intent detection, just knowing what, for instance, the person wants to do. So let's suppose you say, I want to create an alarm for tomorrow. So the intent here is create an alarm. So also to detect a language-- so for instance, if you write among French, you want to detect that text is in French, topic modeling.

The second category is what we call multi-classification. So we still have a text as input, but this time we predict more than one thing. So you have a number of tasks in that bucket as well. So one that is very popular is called Named Entity Recognition, a.k.a. NER. So what that task does is, given an input text, we want to basically label some specific words, like, for instance, identifying whether something is a location, or a time, and so on. And then you have some other tasks as well that are a little bit more on the linguistic side. I think they're less trendy now, but I guess 10 years ago it was something that people would study a lot. So part of speech tagging, which is about just figuring out which word is a noun, a verb, et cetera, or some parsing-related tasks, so dependency or constituency parsing.

And then the last bucket, which is very popular these days is the generation bucket. So you have the text as input, and you also have text as output. And here the length

can be variable, meaning you don't know what the length of your output text will be beforehand. So here you have several tasks. So for instance, you have machine translation. So for instance, something in English and I wanted to let's say German. Question answering-- so typically the ChatGPT, Gemini that you're using, the assistant. So you ask a question and you have a response. And then you have other tasks as well, like summarization, you want to summarize an article, let's say, or just generate something. So something can be generate codes, generate a poem, can also be a lot of things. Cool. So now what we will do is go through these tasks one by one to just illustrate what people typically handle with. So we're going to start with the first bucket, which is the classification bucket. And here we're going to illustrate this with the sentiment extraction task.

So let's suppose we have a sentence, "this teddy bear is so cute." We want our model to predict this to be a positive sentiment. So typically what you would use is sentiment extraction data sets. So I mentioned movie reviews, so this is IMDb critiques. But you also have reviews about products, so Amazon reviews or tweets. Now I guess it's called X, so X posts. And the way you would evaluate such outputs would be by typically using traditional classification metrics. So you have accuracy, which is what is the percentage of the observations that you correctly predicted. But you also have two key metrics, which I'm just going to remind. I'm not sure if everyone knows about them. So one is precision, which is, out of all the positive predictions that you made, which ones were correct? And then the second one is recall. Out of all the true labels, how many of them did you correctly predict as being positive? And you have this metric called the F1 score, which basically takes the harmonic mean of precision and recall to just give you one number. So now you may wonder, why do you need all these metrics? So the short answer is that sometimes you have tasks and data sets where your classes are very imbalanced. So for instance, you can have 99% of your data set that is a positive label, and then only 1% of the data set which is negative. And so here if you take a metric like accuracy, can be very misleading. Because if you have a model that would predict everything as the majority class, then you will have a great classifier, but that's not the case. So that's why precision and recall really play a role.

So that's for the first one. So now let's move to the second category of NLP tasks. So this one is the multi-classification category. So you have an input text and you predict multiple things. And we're illustrating this with the NER task, which as I

mentioned is about identifying the category of given words. And so here, for instance, we want to identify a teddy bear as being an entity. I guess for that, you would use classification metrics, but not at the sentence level, but more either at the token level or at the entity-type level. And by that I mean, let's suppose you have a category, let's say location. And you want to know how well you're predicting words in that category. So you would typically aggregate these metrics as a function of that.

Cool. OK, let's go to the last category, which is, as I mentioned, the most popular one. So this one is text in, text out. So I'm illustrating this with the machine translation task, which is around translating a text from a source language to a target language. So here you have the example with English to French. So cute teddy bear is reading, un ours en peluche mignon lit. So for that, I guess it's harder to get data sets, because here you need to have pairs of texts. So you have a very popular data set that's called WMT, which stands for Workshop on Machine Translation. And that one contains a bunch of paired sequences in different languages. So for instance, you have the English-French, English-German, coming from the European Parliament data set, for instance. So to evaluate those, to evaluate the performance of your model, it's actually a lot more tricky. Because, as you can imagine, you can have many different ways to translate something. I'm sure many of us in the room are bilingual, trilingual. So that's what is making it this hard.
So in the past, people have used several rule-based metrics to do that. So one that you may have heard is BLEU. BLEU stands for Bilingual Evaluation Under Study. And it is a measure of how well your translation stands with respect to a reference text. Same story for ROUGE, which is actually a suite of metrics, but captures that in a different way. And you will see that the machine learning community is funny, because BLEU-- I'm not sure if you know French-- means blue, but rouge means red. So I guess they tried to add some fun in this. But the problem with these metrics is that you always need a reference text. So you basically need labels. And in practice, having labels is very cost expensive. It takes a lot of time, a lot of money to get labels. And we will see later in the class that with the progress that we have made in the LLM space, or that the community has made in the LLM space, we can actually forego of this reference-based metrics and go towards a more reference-free metrics. And we will see that later on. And then the last metric that I will say that people sometimes use is called perplexity. And perplexity only looks at the probabilities that are output by the model. And it basically quantifies how surprised

the model is by its output. So BLEU and ROUGE, the higher the better. Perplexity, the lower the better.

And I guess LLMs have been a hot topic since 2022. But actually, the field goes way back, way before that year. So in the '80s, we'll see it in a second, but there's a class of models that were actually thought of, even in the '80s. And the '90s, we had LSTMs that we'll see also in a second. But the problem was, during that time, we didn't have the internet. We didn't have a lot of compute. And I guess this was one of the limiting factors which prevented the models from today from being trained. And then more recently, we've had several advances. So Word2vec was really one of the pioneering work in just computing meaningful embeddings. And we'll see it in a second. And then, of course, we had the transformers, which were part of a paper that was published in 2017, which is basically at the foundation of all of the models that you see today. And then these models, they just were scaled up, both by compute, but also in terms of the data that was used to train them. And that's how LLMs were dubbed. And I guess these are more like the 2020s. But yeah, I guess we'll see those.

Cool. Any questions on, I guess, the high level? Everyone good? Cool.

Tokenization

So I guess the first question that I want to ask ourselves is, what we want to do is to have a model that handles text. But models, they understand numbers, they don't really understand text. So we need to somehow do something with that text to make it more quantifiable, something that a model can understand. So if you look at a sentence, for instance, "a cute teddy bear is reading," you first need to ask yourself, how can you cut this sentence to pass it to a model? So this part is called tokenization. And what that entails is basically cutting the text with respect to some arbitrary unit of text. So there are several ways of doing this. I guess the first way is doing it completely arbitrarily. So here, for instance, you would have "a." That would be one unit of text. "Cute" could be another unit of text. "Teddy bear" would be another one, and so on. And by the way, the unit of text is called a token, which is why the method is called tokenization. Another way would be to just separate by words. But I guess we would have always pros and cons.

I guess one of the goals that we want to achieve is for us to then be able to represent these tokens in a meaningful way. So one con with doing this at the word level is you will end up with words that look similar, but that are actually considered as different tokens. And I guess the limitation here is you will need to compute embeddings for these similar yet different tokens and somehow make their embeddings similar. So I'll give you an example. So let's suppose I have the word "bear." And then you have another word, plural form "bears." These two words, they are very similar. Just one is singular, the other one is plural. If we go ahead with the word-level tokenization, then we will end up with just two different entities, which are basically just considered as different. Same with "run," and then "runs," variations of verbs.

So for that reason, people have dug into a category of tokenizers that are called subword tokenizers, which is around leveraging roots of words in order to find what are the common roots that we can find in these words. So, for instance, for bear and bears, you would have the bear particle that would be shared. And so I guess the pro is that you get to leverage the root of the words. But then the con here is that your sequence would be longer. And we will see why this is a con. I guess later on, I guess I can give you a preview. So the complexity of these models is also a function of the sequence length. So the more tokens you have to process, the more time it will take for your model to run, because it needs to basically process all these tokens. So that's one con. So pro is it leverages the root of words. Con is it just makes your sequences longer.

You have a last category of ways of tokenizing things, which is just going at the character level, just like taking out characters. So here, I guess, you and I when we write a message, we typically have sometimes misspellings. And with the subword way of tokenizing things, you may not be able to recognize the word that has been misspelled. And this is something that the character-level tokenizer can, I guess, take into consideration. But here the problem is you have a sequence length that's much, much longer, which will make your model, I guess, take much more time to process the sequence. So that's one con. And then the other con is, I guess, when you want to represent each of these tokens, I guess it's very hard to know that a representation of a letter really means. Like, what does the representation of the letter U mean. It's very hard. OK, cool.

So I have just a quick recap. So word-level is a super naive way, super simple way of, I guess, dividing your text into arbitrary units. But then the problem is, as we mentioned, we do not leverage the root of words. And I did not mention this, but there is a term. Whenever you cut something and then at inference time when you want to make a prediction, I guess, one prerequisite that you have is that you need to have the token that you saw at training time, you need to have it in your training sets. And the problem is, let's suppose at inference time you cut your text into words. And let's suppose you have not seen a word at training time. You will need to mark it as unknown. And so this thing is called OOV, Out Of Vocabulary. So luckily, the subword-level tokenizer mitigates that problem. So you have a lower risk of OOV, but still you can have. And as we mentioned, in terms of the pro, you leverage the roots of the words. And then character-level, it's robust to our misspellings and our casing errors. But the problem is it makes computations just much slower. And your sequences would be very, very long, which will also make your, I guess, inference time much higher. That sound good? I guess this is really the foundation, I guess, how to handle things with text. But yeah, does that make sense overall? Cool. OK, so now what we did is we took an input text. What we did is we cut it into parts that are basically tokens.

Word Representation

So in order for our model to understand these tokens, we need to find a representation for each of them. So here, we're going to take a look at this. So that's called a word representation. Or I guess, in a more correct way, it should be token representation. So we want to find a way to represent each of these tokens. So the simple and naive way to do this would be to just assign the one hot vector for each word or for each token. So for instance, let's suppose we have a vocabulary of three tokens-- book, soft, and teddy bears. We would have, let's say, soft. That is 1, 0, 0 vector. Teddy bear, that is, let's say a 0, 1, 0 vector. And book, that is, let's say, a 0, 0, 1 vector. So this is called a One-Hot Encoding, OHE. We'll typically see. So cool. This is a way to represent our tokens. But basically what people want to do compare these tokens to basically see which ones are more similar to what other ones.

So common similarity measure that people use is something called cosine similarity. I'm not sure if you have heard of it. So you can think of it as just seeing what angle these vectors make in the n dimensional space. And if, I guess, they are pointing in

the same direction, then maybe they're similar. Maybe if they're orthogonal, maybe they're independent. And if they're completely opposite, then maybe they're opposite. That's basically the mental model we want to go into. So the problem is, if you represent your tokens in a one hot fashion, you will end up with all your vectors being orthogonal to one another. So that's the problem. So ideally, what we want is for tokens that mean the same or similar to basically have a high similarity. And for tokens that are not similar on about different thing, to be more orthogonal. So here, just for illustrative purposes, teddy bears are soft. So you want teddy bear and soft to be, I guess, with a high similarity. And let's say teddy bear and book, which is independent, you want them to be closer to 0. So that's what you want. That's what you have with one-hot encoding, and that's what you want. Yeah.

Oh, I see. The question is, why do you care about the norm? So I guess cosine similarity is actually normalized by norms. So it's dot products. Oh, you mean why did I just put dot product here instead of 2? Oh, I see. And your question is, why do we not care about the norm? Cool, I guess the viewers know the question. I guess these measures, they are all measures. They are all ways to try to capture these similarity things. So I guess why do you not care about the norm? I guess it's how people have tried to quantify that. I guess you will need to see how your vectors are trained and whether the norm would be indicative of something. I guess the best answer I can give you is, I guess, this is a measure. This is not the perfect measure. People may use also dot product as a measure, but yeah, I don't have a great answer for you. But as long as you capture, I guess, how these vectors they're pointing, I guess, typically what you care about is the angle between them. But typically, you don't really take into consideration the norm. Cool, any questions? Any other questions? Yeah, yeah, yeah, yeah.

It's a great question. So the question is about size of vocabulary and how that would inform the choice with respect to word, subword, and how that changes across languages. So great question. So I would say it really depends, first of all, on the tasks that you're trying to achieve. If your task is just about one language, you will just take that same language. You would typically go with a subword tokenizer just because of the reasons that we mentioned here. So I guess subwords is a nice trade-off between being able to identify words by their roots, like leveraging that, but also running less into the OOV risk. So in terms of the size, I know that people have tried different things. I think typically for English, you would target something on the

order of tens of thousands of vocabulary size. But nowadays, the models, they are multilingual, they are also about codes. So you will see that the vocabulary size now is sometimes on the order of hundreds of thousands. So with respect to Chinese, so I guess you have this difference in characters that you're using. So for Latin, I guess it's the alphabet we're all accustomed to. But of course for the other ones, you'd have something similar, but in I guess the target language character. So yeah, I would say order of magnitude, tens of thousands for one language, hundreds of thousands if it's multilingual. These are the order of magnitude that you want to target for. Cool, yeah.

Great question. So the question is, how do you get those embeddings? So it's actually the next slide. So I'm going to talk about this. Cool, great. So now that we know that the one-hot encoding is not a good way to represent tokens, what we want to do is to learn those embeddings from the data.

So I mentioned that there was this paper that came out in the 2010s-- so I think it was 2013-- that was called Word2vec. And the reason why it was so popular is because they showed a very intuitive and interpretable way of seeing these embeddings, because they were saying something like, OK, king is to queen, what this is to that, like Paris is to France what Berlin is to Germany. So there was basically a way to make sense of the embeddings. So now the question is, how did they do that? So they had two ways of computing these embeddings. So one way was called continuous bag of words. The other one was called skip gram. But they all rely on the same idea, which is let's just leverage texts that we have, and then try to predict something that is part of the text, based on, let's say, the context. So for instance, continuous bag of words, the goal is you take into consideration the words that are around a given target words. And your goal is to predict that target word. And skip gram is the opposite. You go from a target word and you want to predict the words that are around it. So I guess this task is commonly called a proxy task. Because at the end of the day, in this exercise, what we care about is not necessarily to predict the next word, or at least not yet. Our goal is to learn a representation of these words that are meaningful. And so here the idea is, if you have a model that somehow knows how to predict, let's say, the next word, then it means that your model has some understanding of how language works, which is basically what you want. You basically want an embedding that is reflective of, I guess, what language is, which is

king and queen, or similar, Paris and France, this is a capital. You want to have these associations embedded in the representation.

And let's go through a very simple example of what that looks like. So here in our example, let's suppose that our proxy task is about predicting the next word. So here what we take is a very vanilla neural network model, which basically receives a vector of size v, has multiplication and a bias term to get a hidden state, and then another set of multiplications to get our final vector. So here it's basically a very simple neural network. So the input is of size v. The hidden layer is of size d, which is typically much smaller than the vocabulary. So vocabulary is typically like tens of thousands or hundreds of thousands. So d is typically hundred. Like, 768, for instance, is one example of dimension. So it's much, much smaller. So what we're trying to do is to learn the word representation through this proxy task. And what we're going to do is try to consider the words as inputs and predict the next word.

So let's go with the first word of the sequence. So by the way, I use token and words interchangeably. So let's suppose we have the word "a," and we want to predict the next word, which is the word "cute." So what we do is we take the word "a," we take the one-hot encoding representation, and we pass it through the network. So here, if you're familiar with neural networks, so here you have, I guess, a multiplication between a matrix and this vector. So you have a hidden state representation, which is a vector of size d. So here, let's suppose it's 0.2 and 0.9, so D equal 2. And then you have, I guess, another pass here. And then you get, after softmax, a set of probabilities which are around seeing what is the next word. So in this example, we have a vocabulary of size 6. So the first word is predicted with probability 0.2, second word, 0.4, and then the other words are all 0.1 in this example. So let's suppose that we want to somehow be able to maximize our prediction to be the second word of the vocabulary, which is the 0.4. So we basically compare the prediction with, I guess, 0, 1, 0, 0, 0, which is the representation of the second word of the vocabulary. And then we do the back prop, we update the weights. I'm not sure if everyone is familiar with that part. But the idea here is, once you obtain a prediction, you compute the loss, so typically cross-entropy, which will determine how far off you are from the true answer. And based on that difference, you're going to update the weights in order to make your prediction closer to the truth.

So that's what you do. And then you repeat that process. Let's suppose you take the word "cute," which, as we said, is the second word in the vocabulary. So the one-hot encoding representation is 0, 1, 0, 0, 0. So you go through that network, you have a hidden state, like the vector is 0.8 and 0.4. You do that again. And what you want to do is to predict the next token, and here is teddy bear. And so you see now your model in this example is predicting the next word to be uniform, but you want to somehow maximize the probability for teddy bear. So you go about doing this again and again for all the words. And at the end of the day, you obtain a model that learns how to predict the next word, which is basically the proxy task. And what you're going to do is to take the representation that the model learns, which is the green units. So what happens now is every time you have a word, you just represent that as a one-hot encoding representation. And you just multiply this with these weights, and then you obtain the green representation. And that is your word representation.

Yeah, great question, great question. So the question is about what does v correspond to and why there's only six. So yes, in this example we only have six possible words, which is basically the vocabulary size, just like a very toy example, because in practice there is many more. So I guess that's one of the challenges with language. So you can technically have many variations of words, which is why if you take a word-level way to divide your text into tokens, you can end up with the vocabulary that's very big, because you need to account for all the variations of given words. And the other thing that I want to point out is, let's suppose you have a vocabulary size of 6, and it's the six words that you saw at training time. But what happens if at inference time you have a word that you have not seen at training time? And so the answer for that is typically what people do is they reserve a spot for what they call an unknown token or out-of-vocabulary token, which is basically you can think of it as a bucket for everything that we were not able to identify. So if let's suppose at inference time you have a token that you were not able to identify, they will all take that representation, which is the unknown token representation. And this is, by the way, something that I guess the word-level tokenizer has trouble to do, because you will have a much bigger chance of having out-of-vocabulary tokens. Subword level will have a lower chance. And then character level, I guess, you don't have that problem. Does that answer your question? Yeah. Cool, yeah.

Great, great question. So first question is, when you're done? So the thing with the proxy task is when you train your model, I guess your true objective is to not really

learn-- I mean, in this case-- to learn how to predict the next word. Your objective is to have meaningful representations. But what you can do is to somehow track the loss function for the proxy task that you're pursuing, but then also taking into consideration that this is not necessarily your end goal. So I guess one very reasonable way of going about doing this is just to wait until your model converges. So here, what you do is you track the loss as a function of-- so there's this term "epoch," just how many times your model sees the training set. And so you compare these different curves. And when this converges, this is typically a good time to stop the training process and just see if that makes sense. Depends on your downstream task, of course. But that's one.

So your second question-- sorry, can you repeat the second question? Yeah, yeah, oh, great question. So the question is, how do you know when the generation stops? I guess, otherwise it will never stop. So yeah, exactly. So you have some special tokens. Typically, you have end of sequence, end of sequence. So typically when you have the end of sequence token generated, then it's when it stops. All right.
So second question was what informs the size of the hidden layer? I would say it's a trade-off, because you want the embedding to be rich enough that it can be informative for your downstream task. So for instance, if you want to somehow get an embedding of, let's say, your sentence, and if you want to, let's say, do a very, very specialized task, like with a lot of different outcomes, maybe you want a vector that recaptures that, so maybe you want a bigger vector, but if you had a very simple task, maybe a smaller vector might make sense. So I guess the size of your hidden dimension also impacts the complexity of whatever you're running after. Because of course, if you have longer vectors, you'll have more computation, so your inference will be probably more expensive, et cetera. So I guess there's a lot of factors. So I guess just to recap, one is how complicated your downstream task is. Second one is how sensitive are you with latency, cost, all these things. So it's really a trade-off. But out there you would typically see embeddings of a pool of hundreds or thousands. Of course, these models, they've been growing, so this number may change. But that's the order of magnitude that you're looking at. Right. This is indeed empirical. Yeah, yeah. I guess you can also rely on what others found and just go from that. But 768, these numbers are things that people typically take. Cool, yeah.

Great question. So the question is, how can you distinguish words that are spelled the same but in different contexts? So you're way ahead of me. So this is basically the basics. And we're going to tackle methods that can tackle these problems of just

contextualizing the word in the sentence. So yeah, so we'll see that in a bit. Cool. I'm not on time. So I'll try to get moving.

Recurrent Neural Networks (RNNs)

So OK, so now what we did was see how we could learn representations of tokens. But I guess you may also want to get representations of sentences or pieces of text. So one very naive way to do that with what we saw before is to take something like the average of words, let's say, the word representations. But the problem is you lose a lot of meaning, you lose the order, you lose-- and I guess here, I think you pointed out very well, the representations that you learn are token-specific, regardless of where they're at.

So that's why we have a class of models that aim at capturing the sequential nature of how text appears. So we're going to talk about RNNs, which stands for Recurrent Neural Network. So what RNNs do is, instead of processing words one at a time, what they do is they keep a hidden representation of the sentence so far, and they consider tokens one at a time. So as I mentioned before, this technique was actually introduced a fair amount of time ago, so in the '80s. And what this model does is it takes into consideration the order at which words appeared or tokens appeared. And so in this example, you start the, I guess, processing at the very beginning of the sentence. You have some dummy hidden states that is called A, typically denoted A or H. It's called the hidden state, activation, or even sometimes a context vector. And you have some kind of a module that takes into account the hidden state so far and the word at time step t, so here time step 1. So here what it does is it takes in the meaning of the sentence so far and takes into consideration the word that is happening now. And it produces an output vector that here can be used to try to predict the next word. So for instance here we have this hidden state and the representation of the word that then you have some matrix multiplications in this blue box. And you have an output vector that you try to train on predicting the next words. And then you keep on doing that by keeping track of these hidden states. And so you repeat the process. And I guess the way you would interpret these hidden states is it's a representation of the sequence process so far. So the good thing with RNNs is now the word order matters. And you're also able to encode the sentence in a more natural way. So let's see roughly how it works.

So we have the same favorite example, so cute teddy bear is reading. So you would have the token A. You want one-hot encoding vector. You pass it through your network. You compute the hidden state. You try to predict cute. But then you keep track of the hidden state. And then you input that into another module. And then you also consider the next words. So you consider not only the word itself, but also the hidden state of this sentence so far. And you try to predict the next word again, and again and again. So this is RNN. So RNNs were used for a bunch of tasks, and just like mapping that back to the categories that we saw before. For classification purposes, you can basically use the hidden state of the last word in your sentence. For instance, if you want to predict like the sentiment of a review, you would take basically the last vector here and try to project it into the space of the predictions or the labels that you want to predict on. So for instance, if you want positive or negative, you basically project that vector onto that space. You can do that here. For multi-classification, so you would basically have the representation of the token of interest, and you would project that. Or for generation, you would basically process the whole source text, and then have a context vector, a.k.a. activation vector, a.k.a. hidden state at the end of your processing, which will then be used to decode the output prediction. So this is how you would use an RNN for each of these tasks.

So the reason why you have not really heard of RNNs these days is because they had some pros, but a lot of cons. So one of the cons is that the meaning of the sentence is basically solely encapsulated into this hidden state. So you have this problem of long-range dependencies, which basically impacts your ability to quote, unquote, "remember" what the model saw in the past, which is why you have another class of models that try to build on RNNs. So this one is called LSTMs, Long Short-Term Memory. And the goal of that extension is to have the way to somehow keep track of the things that are quote, unquote, "important" to remember, on top of the hidden state that we talked about. So here you have a of t, which is your activation, basically the sequence so far encoded in there. And then you have another quantity that you track that is called the cell state. It's denoted c here. So this architecture aims at improving that piece, but I guess it was not perfect either. But yeah, so that was the main issue of RNN-based methods, which is that they have this issue of forgetting what was in the past.

So you will see in the literature that this phenomenon is called vanishing gradient. And the reason why it's called that way-- so I know we're running out of time, but I'm

going to just explain that part. So in order for you to predict, let's say, the last words, you're basically dependent on every hidden state that came before that. So far so good? And so whenever you want to update the weights of your model to match the prediction here with the actual prediction, when you do the back propagation, you somehow need to take into account that the value here is basically not only a matter of this computation, but also this computation or this computation that basically happened in a sequential manner. So you have this phenomenon of trying to, I guess, back-propagate through time. But the problem is, in practice when you write that down-- so it's a very ugly formula. But when you write that down, it ends up being a product of a bunch of quantities that can-- so if it's greater than 1, then it's exploding. If it's less than 1, it's vanishing. Because if you multiply a lot of things that are less than 1, it just goes to 0. So I guess if you have something that you're trying to update that goes to 0, basically you have trouble just doing your updates. So that's a high-level intuition. This is not the focus of this class, which is why I'm not going into the detail of this ugly formulas. But I hope you get the idea that for remembering things from the past, it's not doing a great job, because of this sequential, I guess, characteristic. Does that make sense?

OK, I hope the next thing will make a bit more sense. But before that, I'll just recap what we saw. Our goal is to represent text. So we first started with representing words or tokens, which was what we tried to do with Word2vec. And we saw that it was a good way to leverage proxy tasks to learn this representation. But we had a bunch of limitations. And one of them that you mentioned was that this was not aware of the context. And also the word order didn't count. And so you have this other class of methods that is able to take into consideration the words, but then they have some trouble keeping track of things when the sequence gets very long. And you have this problem of vanishing gradients or long-range dependencies. So whenever you see this term, it's basically referring to that. And also another thing that I have not mentioned, but the computations are very slow. So when you want to train these models, at training time, in order to predict this word, you basically need to compute all these hidden states before. So when your sequence gets very long, it just takes a very long time.

So for all of these reasons, for I guess what reason. So for the fact that the model has trouble remembering things from the past, people have tried having more direct connections between something and the thing from the past, and this is the idea

behind attention.
So what attention does is it tries to have a direct link between what we're trying to predict and something from the past. So in this example, let's suppose I'm trying to translate an English sentence into a French one. So here I guess the input sentence is given. I'm computing the hidden state. I'm processing words one at a time. This is my traditional RNN. So "a cute teddy bear is reading." So here I have a hidden state that I'm then decoding. And you can imagine that when wanting to generate the next word of my translation, it would be great if I knew what word I'm trying to predict. Or in other words, it would be great if I could take a peek at a certain area of the input text. So the idea behind attention is to have a direct link between what you're trying to predict and things before. This is the idea behind attention. And so it was introduced in 2014. And yeah, again, this is trying to solve for these long-range dependency issues. And so yeah, this example we want to do that. And this concept is going to actually be key for this class.

Self-Attention Mechanism

Because we're going to see that the attention mechanism is the thing that is going to make everything-- I mean, most of the things work. And this is actually the main principle that the transformer paper relies on. So the transformer, which is the core architecture that we will see in this class, has been introduced or was introduced in 2017 in this paper named Attention is All You Need. So even from the title, you can see that the authors wanted to just rely on that part. So what the authors tried to do was to move away from this sequential way of processing the text, and instead let the model just have direct connections with all parts of the text at once. So that is called self-attention. So they tried that on translation tasks. And they just realized that it was giving great results.

So back to the example that we're still using, a cute teddy bear is reading Here what we would say is that in order to compute the representation of the token teddy bear, we're going to look at all the other tokens in the sequence at once, and directly with direct links. So I guess back to your question-- here, we would have a representation of teddy bear that would be unique to the context that it is part of. So back to your question about riverbank and robbing a bank, like here the bank would have different representations. So this is the idea. I guess, does the idea roughly make sense? And again, this is called the self-attention mechanism. Afshine, how am I doing on time? 8. I have 8? OK, cool. OK, so this is the idea. So now I'm going to just introduce

another set of ideas which is more terminology but is going to be very important. So when you want to express something in terms of something else, we use the words "query," "key," and "value," Q, K and V. So in this example, our goal is to figure out what other tokens is the query teddy bear more similar to. So here the question is, OK, you have a query and you want to see what other tokens are most similar. And so what you're going to do is to look at all the other tokens which are basically composed of keys and values. So we're going to compare the query to the key to quantify how similar your query is to a given key, and take the corresponding value. So we'll see that in this example, so let's suppose you want to express teddy bear in terms of everything else. What you're going to do is you're going to take the query teddy bear, and you're going to compare that query with all the other keys to see which element is most similar, and then weight the more similar ones and take their associated value. So that's a very high level idea of how these things are. Of course, we're going to see exactly how they work. But that's the general idea.

OK, cool. And speaking of query and key and value, we will also see that one benefit of expressing things this way is that we can express doing this self-attention computation across the whole sequence in a matrix format. And GPUs love matrices. So it's really made for the hardware that we have. And I guess what I mentioned here can be expressed in a form of softmax of the query and the key, which is basically a way to get some kinds of weights of which values will be more important. So for instance, if a value is more important, you have bigger weight. And another one will be less important, you'll have a smaller weight. And you basically multiply that by the value. So don't worry, we'll have a detailed example after. So if it still feels very high-level fuzzy, don't worry. We'll have a detailed walkthrough. And yes, so this is how it works. OK, cool. Any questions on what self-attention is? Yeah. Great, great question. So the question is, what is value, what is key? I guess, how to get those? What do they mean? So first of all, I just want to say that these quantities, they are learned. So you are not fixing them. But from an interpretation standpoint, you can interpret that the key is there for you to figure out which one is most similar to the query. And the value is the actual value that is associated with that element. So here, you will have something like you want to express this in terms of all the values. So the weights in your weighted average will be basically the dot product between-- basically-- between the query and the key. And the value will be the actual vector that you will use. But again, these things are learned. And something I have not mentioned, but you mentioned it correctly, so we're going to actually do projections to obtain these quantities. And these projections are actually learned by

the model. That's good? OK, cool. So with that, we have 15 minutes, right, to talk about the architecture.

Transformer Architecture

OK, so at a very high level, in order to make the self-attention mechanism happen, the authors propose an architecture that is composed of two parts, an encoder, which is on the left side, and a decoder, which is on the right side. So the application that they have is translation. So what will go through the encoder is the input text in your source language. And what is going to go through the decoder is the target language that you're predicting. So the high-level idea is you're going to compute meaningful embeddings from your input text by passing them through the encoder. And you want that self-attention mechanism to apply, meaning you want to compute representations of each token as a function of others. And you do that by using a layer called the attention layer.

So multi-head attention layer, but multi-head is just doing this computation in different ways to just allow the model to learn different representations or different projections. But the idea here is you are going to input your input text, and all the tokens in your input text are going to attend to one another. So for instance, a cute teddy bear is reading, you are going to compute the representation of all the tokens in this text basically as a function of others. And you're going to do that with the encoder, so here with the multi-head attention. And then you have a feedforward layer, which is just to let the model learn another kind of projection. And what you're going to obtain at the end of your encoding process is rich representations of the tokens from the input sentence. So far, so good?

But now your goal is to actually translate the input sentence. So what you're going to do is to start your translation with, let's suppose, the beginning of sentence token, so your first token. And what you're going to do is use all the representations from your input sentence in order to figure out what to predict next. So this what I just said is the cross-attention layer, which is the one that is the second, so this one, which basically-- I'm not sure if you see the arrows, but there are two arrows coming from the encoder, one arrow coming from the decoder. Can anyone tell me what the arrow from the decoder represents? Is decoder a query key or value? I guess there is 1 over 3, 33% chance. Who wants to try? Is it key? OK.

Query? OK. So the way to think about it is you're trying to ask yourself, what are the words from the input that matter. Right? So basically, you want know, given your

query, what are the elements from the inputs that matter? So here, this arrow is indeed the query, because this is the thing that you want to figure out. And the keys and values are actually coming from the encoder, which are basically coming from the input sequence. And then you have another attention layer, which is this one. And that one is trying to figure out what other tokens of the output sentence that you're decoding is going to be useful to predict the next token. So let's suppose you start decoding and you say, un ours en peluche, which is in French. To predict the next word, you want to basically figure out what are the tokens translated so far that are going to be useful to predict the next word.

So this is what this attention layer is about. And it's called masked, because it only looks at the tokens that translated so far. It does not look at tokens that were not translated, because of course they were not translated. So there's no way, like on the right side of the token that you're trying to predict. Cool. So at a very high level, you have this attention layer, which is present in the encoder, which is present in the decoder, but it has several, I guess, use cases. So the attention layer here aims at computing embeddings from the input sentence as a function of themselves.

And then the ones from the decoder, so the first one, the masked self-attention layer, aims at expressing something as a function of everything that has been decoded so far. And the second one, the cross-attention layer, tries to express things as a function of what has been seen in the input. So here, given that you are having direct links to different tokens, you don't have this sense of order. Because in the RNN, you are basically expressing things one at a time. So you had some sense of the word order, but here you don't have it, because it's like a direct link, which is why you have position encodings, which are there to inform on the position of the word in the sequence. So we're not going to dig into that today, but I just want to call that out.

So at a very high level, and we're going to see this in the detailed example, what we do is in order to translate a sentence from source language to target language, we're first going to tokenize the text, so dividing into arbitrary units. We're going to learn an embedding for these tokens. So this is what the input embedding is about. Then we're going to add some encoding with respect to the position. We're not going to talk about it today, but just good to note.

And then we go through the encoder. So the encoder tries to figure out how to express things as a function of other things from the inputs. So it does that in the multi-head attention layer. And then it goes through a feedforward neural network, which is just a way to just project the vectors to just have some more degrees of freedom to learn things. And then once you have these representations from the

input, you're then going to start your translation. So you start with the BOS token. And what you're trying to do is figure out what the next word is. So you're going to see, OK, what are the words that were translated so far that are useful for translation. So this is what the masked multi-head attention layer does. And then you have another attention layer, which is about expressing things as a function of what was in the input, which is the cross-attention layer over there. And then you have a feedforward neural network to, again, give some more degrees of freedom. And at the end of the day, you have a vector that you then go through softmax. And it just is a way for you to guess what is the next word. So you have a vector of vocabulary size. And you're going to use these values to determine what is your next word. Easy, right? Any questions on this? Yeah.

All right, that's a great question. The question is, what does head mean? So I guess I went too fast. I ignored that part. But when you do the self-attention computation, you basically make queries interact with keys, and then take the corresponding value. But nothing prevents you from doing that several times. So the term "head" is given to the projection matrices that you use to obtain the query, key, and value. And when you have several heads, what you're doing is you're allowing your model to learn different projections. So it's basically an additional degree of freedom for your model to learn different associations between your vectors. So it's a great question. So typically it will be noted lowercase h, number of heads. And this is what this corresponds to. Does that answer your question? Cool, very cool.

OK, we have a lot to discuss, but here I had the slide actually for this. So this is the multi-heads that you were mentioning. So we're basically running the self-attention computation several times in parallel, again, with different projection matrices that the model learns. So in case you have a computer vision background, it is similar to having multiple filters in your convolution. So it's the same idea, but it's different here. Yeah, great question. So the question is, are the projections different? So we're typically not constraining things. We're just letting the model learn. But in practice, it just tends to learn different ways of saying the same thing. So yeah, typically there is no constraint. Of course, you have papers that dig into how about if you change this. But typically you don't have any constraint. Cool, great.

OK, I will just mention one trick, another trick that the transformer authors use. So it's called label smoothing. Who has heard of label smoothing? So one new thing here is in NLP when you want to predict what comes next, there's typically more than one way. When you say, what a great day, what a great lecture, what a great book, what a great-- there's always multiple choices. There's more than one way of filling that gap.

So label smoothing is a technique that tries to intuitively address that. And what it does is, instead of saying predict this word 100% is this one, there is no other words, what it does is it says, predict this word, but there's a chance it's not this word. And in practice, what it does is it takes the one-hot encoding. And instead of saying it's a 1, 0, 0, 0, that you need to predict, it says it's actually 1 minus epsilon, and then epsilon over v minus 1, I guess, is you're trying to predict. So in practice, it's a method that tends to make your model be more unsure. Again, to be less sure about this prediction because you always tell it, OK, try to predict this, but actually it's possible it's not the correct value.
But in practice, the authors see that it tends to improve metrics like BLEU, which is a proxy metric for translation tasks. So yeah, I think this method is pretty general for NLP. So yeah, it's a good one to know. And with that, I think there's about 20-ish minutes left. So yeah, Shervine is going to walk you through an end-to-end example. And with that, you-- yeah, Oh, u? So I guess here you can think of this as, I guess, some quantity. It's not defined, so some quantity. And the delta is like one hot, if you want, something like this. It can also be a constant. Yeah, yeah.
So question is, is there a relation with explore and exploit? It's an interesting one. So why would softmax give it for free, by the way? Right, but I guess it would still-- so I guess at the end of the day, what you're trying to do is to compare your prediction with respect to the label. So I guess the question here is, do you want to compare with 1, 0, 0, 0, or do you want to compare with something that is not 1, 0, 0, 0. So I guess softmax does not allow you to do that. Exactly, yes. So I'm not sure if this was super clear, but this is actually the label. So what you're trying to predict is not 1, 0, 0. But we change the label in a way that makes the model, I guess, predict something that is less sure, I guess. Cool, thanks. And with that, yeah, Shervine. OK, great. Thank you, Afshine. And yes, so we saw basically how the transformer worked. And now we're going to piece it all together with the one specific example.

Detailed Example

OK, great. So let's take our favorite example again. So a cute teddy bear is reading. And then we'll go all together through each step. So first we start with tokenization. So as we said, we can use any arbitrary decomposition to decompose this into tokens. And then as someone mentioned, you need to have some way to indicate the start and the end of a sequence. So typically, this is done with the BOS and EOS tokens. So you add them.

So now let's focus on the composition of each token representation. So you have its embedding. That is learned. And as Afshine mentioned, in order to have an idea of what is the position of the words or the token, I should say as part of the sequence, you have some added information that is in the form of a position embedding. And here, the original paper uses the convention of some sines and cosines that it adds additively to the representation. So it's like an element-wise addition. OK, great. So now you have a position-aware embedding for your token. And you repeat that for each of your tokens. So now you can see all of these embeddings in the format of a matrix, which is of size d model, which is the size of your embeddings. And then the other dimension is the length of the sequence, so typically n. So all make sense so far? Any questions on the inputs? OK, great. So now we will send this representation through the encoder. So as Afshine said, you have this concept of self-attention. And the way you perform self-attention is that you take this input and project it on three spaces.

So you project it to the space Wq, you get queries. You project the same embeddings in the space Wk, you get keys. And you do the same for values, you get your values. And then Wq, Wk, and Wv are learned by the model. They are basically projection matrices. So far, so good? So now with all of that in mind, you can apply the formula that Afshine mentioned, that is the self-attention formula, which is softmax of Qk transpose over square root of dk times v, which gives you another matrix out of all of this. Now let me pause for a second and look at how this computation is done in practice and what every step means.

So let's look at Q. When you compute Q, basically you project your embeddings into that space. What do you obtain? You obtain a matrix where each row represents a given query. When you say k transpose, it's basically the same matrix, but transposed, where each column represents the key representation of each token. Now let me mix them together with the matrix multiplication. So when you multiply each of them, you see that each row represents the projection of the query over each key, such that when you take the matrix multiplication and get the softmax of all of this, you get a probability distribution of the projection of the query over keys for each query. Each line will have this. And I don't know if anyone asked the question regarding why do we scale by square root of dk? So it could be dq as well because matrix multiplication, like the dot product here, enforces the fact that dq equals dk. And basically, what you see is that these dot products has the dimension of key and queries grows, it will tend to grow as well. So you want to normalize these dot products. And this is why you divide by square root of the dimension of keys. OK,

great. And then now you have your softmax of all of these, and then you multiply it with the matrix v. And this is Afshine explained as having the query projected on the space of keys, and then multiplied by the corresponding value. So the value is the representation of the corresponding key that we project on. So you end up with a weighted sum of values for each query.

OK, great. Does that make sense so far? OK, awesome. And someone asked, what is the multi-head stuff? So you're right, it's not just single one, one time that it's done. It's actually done each times. And what you obtain is, like, all of that is done in parallel. And at the end, you obtain h such matrices. And you concatenate them with respect to the columns. And at the end of this, you have another projection matrix that you call Wo that will project all of these back to the original dimension of embeddings. So it's a way for the network to basically have a dimension-invariant way-- bless you-- to go from the original dimension back to the original one. Any questions? Yep.

So the question is regarding h. Is it possible to get the same result each time? And if so, concatenate the same thing, will it be helpful? Did I get the question right? So what makes it different? So it's the magic of gradient descent. So the network has an objective function at the end. It has degrees of freedom. Its incentive is to build a representation that will be helpful to learn the next word. So it doesn't have an incentive to copy the same thing or do the same mechanism. And this is why in practice you see the model converge towards building different representations that it can then concatenate and then project into something useful. So what makes it such that you don't have the same thing? Nothing. Like, you don't have any constraints. But the nature of the learning that you let the model have makes it do so in practice. Any other questions? And that is a great question. I mean, typically gradient descent does wonders.

OK great. So now that we have gone through the self-attention layer, you have another component that is the FFN. And I think there was a question just here regarding how to choose the dimension of the hidden layer with respect to the input and outputs. So when Afshine mentioned Word2vec, typically you have a smaller dimension than the input and output. But here, actually, the hidden layer is of a bigger dimension than input and output. And the rationale for that is that you want to have enough degrees of freedom for the model to learn useful representations. So it's a way to complexify the features that you learn. And, yeah. OK, great. And you don't have just one encoder module. You have actually n of them, big N in the original paper. And at the end of all of this, you have an encoded context-aware set of

embeddings. And then each of these setups encoded embeddings will be those that will be fed to the n decoders. So you have like a stacked succession of n encoders. You have a stacked succession of n decoders. And the last representation of the encoder is what you will feed to the cross-attention of each decoder. So yeah, we're going to see that more in detail.

So how do we even start the decoding process? So you start with the BOS token. Basically saying to the model, hey, we need to predict the next word, let's start. So what happens to the BOS token at the very beginning? So you feed it to the decoder. And then similarly as before for the encoder, you have a self-attention layer. And as Afshine mentioned, the self-attention layer is causal. So the attention will be done on the same token and the tokens that precede it.

So on this first BOS token, you don't see a difference, because it will just attend to itself. But when you have other tokens that you want to decode, you will have this difference in where you attend with respect to decoder. OK, great, goes through the self-attention layer. And then you have what I mentioned to be the cross-attention that takes as keys and values these encoded embeddings as inputs. And then the queries are those that come out of the self-attention layer. OK, great. And then once you do this cross-attention, you have, just like in the encoder, an FFN component that makes the representation richer. Was there a question there? No.

And then at the very end, so you do all of that n times. And at the very end of the decoding process, you have a linear projection and a softmax layer to turn the prediction of the next word into a probability distribution over the vocabulary. OK, great. So we saw how to do that for the next word here. And basically you do that again and again. So you have found your next token, which is like a one-hot basically encoding of what you want. And then you take that embedding and then put it back in the decoder and continue this process. And when do you stop? It's a question for you all. When you hit the EOS token. Yeah, yeah exactly. OK, great. And with this process, it's basically how the authors of this original landmark paper did machine translation. So this is typically the use case that was presented.

OK, awesome. And with that, thank you for your attention. [APPLAUSE]